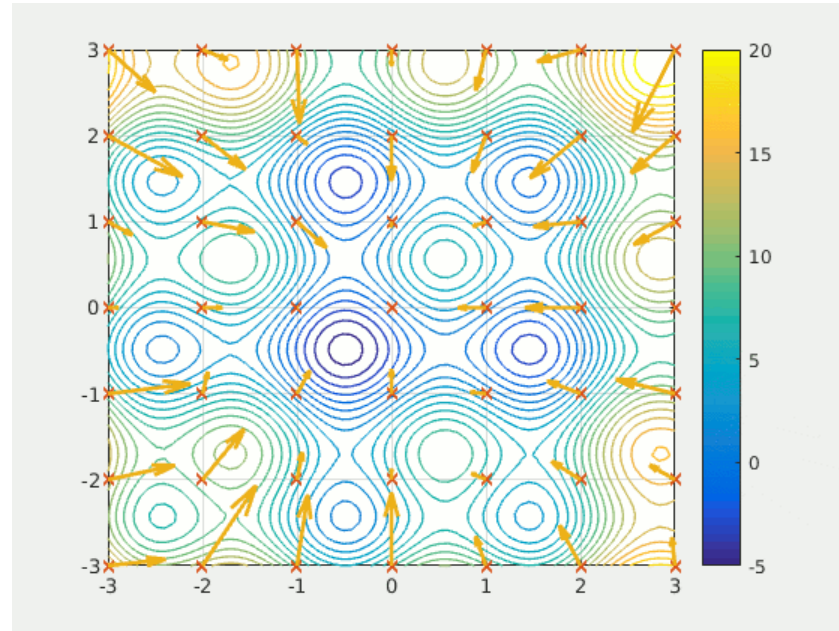


Programowanie równoległe i rozproszone

Metoda optymalizacji rojem cząstek (PSO)



Źródło : https://en.wikipedia.org/wiki/Particle_swarm_optimization

Autorzy: Mateusz Chmurzyński, Filip Kiernozek
Bartłomiej Krajewski, Bartosz Sędzikowski

Zadanie

Wykorzystane technologie:

1. Wersja sekwencyjna
2. Procesy
3. Wątki
4. CUDA
5. MPI

Język programowania: Python

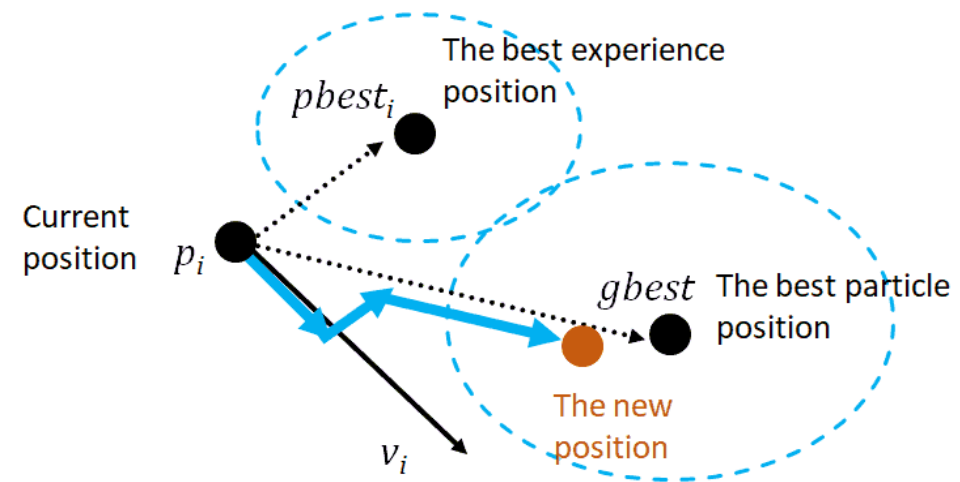


Źródło : https://en.wikipedia.org/wiki/Python_%28programming_language%29

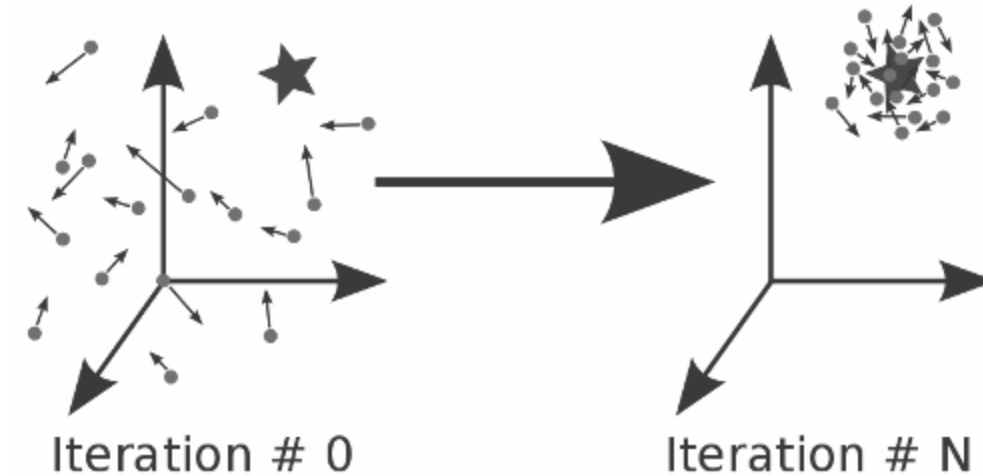
Metoda optymalizacji rojem cząstek (PSO)

$$v_i(t + 1) = wv_i(t) + c_1r_1(pbest_i - x_i(t)) + c_2r_2(gbest - x_i(t))$$

$$x_i(t + 1) = x_i(t) + v_i(t + 1)$$



Źródło : <https://www.baeldung.com/cs/psa>



Źródło : <https://esa.github.io/pagmo2/index.html>

Procesy

- Python + Docker
- Biblioteka *multiprocessing*
- Podział wejścia do funkcji celu na kawałki (*chunks*)
- Funkcja celu wykonywana wykonywana równolegle
- Łączenie danych sekwencyjne

Procesy

6.3.1 Zadanie 1: Funkcja Schwefel

Tabela 5: Wyniki dla funkcji Schwefela (Swarm Size = 1000)

Wymiar (n)	Kryterium	$i_{max,proc}$	Najlepsze $f(x)$	$T_{proc,2}$ [s]	$T_{proc,4}$ [s]	$S(n, 2)$	$S(n, 4)$
10	Znane Opt.	120	$1.27 \cdot 10^{-4}$	0.27	0.32	0.32	0.27
10	Brak Popr.	161	$1.27 \cdot 10^{-4}$	0.76	0.32	0.13	0.30
50	Znane Opt.*	10000	6726.75	40.43	45.59	0.76	0.67
50	Brak Popr.	357	6726.75	1.31	2.34	0.79	0.44
100	Znane Opt.*	10000	11854.13	75.51	76.44	0.74	0.73
100	Brak Popr.	1024	11854.13	7.72	7.75	0.77	0.77

*Algorytm osiągnął limit iteracji nie znajdując optimum z zadaną dokładnością.

6.3.2 Zadanie 2: Funkcja Rosenbrock

Tabela 6: Wyniki dla funkcji Rosenbrocka (Swarm Size = 1000)

Wymiar (n)	Kryterium	$i_{max,proc}$	Najlepsze $f(x)$	$T_{proc,2}$ [s]	$T_{proc,4}$ [s]	$S(n, 2)$	$S(n, 4)$
10	Znane Opt.*	10000	5.42	12.96	20.97	0.32	0.20
10	Brak Popr.	191	5.42	0.27	0.39	0.30	0.20
50	Znane Opt.*	10000	115.20	32.68	39.19	0.66	0.55
50	Brak Popr.	9003	115.21	27.74	37.78	0.70	0.52
100	Znane Opt.*	10000	119.96	70.40	77.37	0.65	0.59
100	Brak Popr.*	10000	119.96	67.22	74.23	0.63	0.57

*Algorytm osiągnął limit iteracji.

Procesy – etap 1

6.1.1 Zadanie 1: Funkcja Schwefel

Tabela 1: Wyniki dla funkcji Schwefela (Swarm Size = 1000)

Wymiar (n)	Kryterium	Iteracje	Najlepsze $f(x)$	T_{seq} [s]	T_{par} [s]	Przyspieszenie (S)
10	Znane Opt.	120	$1.27 \cdot 10^{-4}$	0.96 s	0.75 s	1.28
10	Brak Popr.	161	$1.27 \cdot 10^{-4}$	1.22 s	0.86 s	1.42
50	Znane Opt.*	10000	6726.75	90.95 s	47.96 s	1.90
50	Brak Popr.	357	6726.75	3.61 s	2.87 s	1.26
100	Znane Opt.*	10000	11854.13	114.80 s	67.20 s	1.71
100	Brak Popr.	1024	11854.13	13.06 s	10.48 s	1.25

*Algorytm osiągnął limit iteracji nie znajdując optimum z zadaną dokładnością.

6.1.2 Zadanie 2: Funkcja Rosenbrock

Tabela 2: Wyniki dla funkcji Rosenbrocka (Swarm Size = 1000)

Wymiar (n)	Kryterium	Iteracje	Najlepsze $f(x)$	T_{seq} [s]	T_{par} [s]	Przyspieszenie (S)
10	Znane Opt.*	10000	5.42	99.16 s	39.12 s	2.53
10	Brak Popr.	191	5.42	1.99 s	1.03 s	1.93
50	Znane Opt.*	10000	115.20	110.38 s	54.96 s	2.01
50	Brak Popr.	9003	115.21	101.85 s	54.14 s	1.88
100	Znane Opt.*	10000	119.96	138.10 s	97.86 s	1.41
100	Brak Popr.*	10000	119.96	150.53 s	106.02 s	1.42

*Algorytm osiągnął limit iteracji.

Wątki

- Python 13.3
- --disable-gil --enable-optimizations
- import threading

- barrier_start = threading.Barrier(n_threads + 1)
barrier_end = threading.Barrier(n_threads + 1)

- def pso_worker(...):
...
barrier_start.wait()
...
barrier_end.wait()

- stop_event = threading.Event()

- t = threading.Thread(
target=pso_worker,
args=(...),
daemon=True,
)

Wątki

Tabela 7: Wyniki dla funkcji Schwefela (Swarm Size = 1000, wątki=2)

Wymiar (n)	Kryterium	$i_{max,2}$	Najlepsze $f(x)$	$T_{thr,2}$ [s]	$T_{thr,2}/i_{max,2}$ [s]	$S(n,2)$
10	Znane Opt.	4074	145.83	11.85	$5.81 \cdot 10^{-2}$	0.01
10	Brak Popr.	167	145.83	8.68	$5.20 \cdot 10^{-2}$	0.01
50	Znane Opt.*	10000	6046.14	36.17	$3.62 \cdot 10^{-3}$	0.85
50	Brak Popr.	375	5855.81	11.90	$3.18 \cdot 10^{-2}$	0.09
100	Znane Opt.*	10000	15217.20	49.57	$4.96 \cdot 10^{-2}$	1.12
100	Brak Popr.	852	15715.70	13.81	$1.63 \cdot 10^{-2}$	0.36

*Algorytm osiągnął limit iteracji nie znajdując optimum z zadaną dokładnością.

Tabela 8: Wyniki dla funkcji Schwefela (Swarm Size = 1000, wątki=4)

Wymiar (n)	Kryterium	$i_{max,4}$	Najlepsze $f(x)$	$T_{thr,4}$ [s]	$T_{thr,4}/i_{max,4}$ [s]	$S(n,4)$
10	Znane Opt.	2110	72.91	12.29	$6.21 \cdot 10^{-2}$	0.01
10	Brak Popr.	164	72.92	8.94	$5.41 \cdot 10^{-2}$	0.01
50	Znane Opt.*	10000	5956.52	25.10	$2.51 \cdot 10^{-3}$	1.22
50	Brak Popr.	408	25.02	11.13	$2.73 \cdot 10^{-2}$	0.11
100	Znane Opt.*	10000	15580.81	37.92	$3.79 \cdot 10^{-3}$	1.47
100	Brak Popr.	795	15237.43	6.47	$8.53 \cdot 10^{-3}$	0.68

*Algorytm osiągnął limit iteracji nie znajdując optimum z zadaną dokładnością.

Wątki

Tabela 9: Wyniki dla funkcji Rosenbrocka (Swarm Size = 1000, wątki=2)

Wymiar (n)	Kryterium	$i_{max,2}$	Najlepsze $f(x)$	$T_{thr,2}$ [s]	$T_{thr,2}/i_{max}$ [s]	$S(n, 2)$
10	Znane Opt.	6953	$5.18 \cdot 10^{-7}$	13.58	$2.56 \cdot 10^{-3}$	0.16
10	Brak Popr.	3020	0.80	13.07	$4.63 \cdot 10^{-3}$	0.09
50	Znane Opt.*	10000	2268.32	27.83	$2.78 \cdot 10^{-3}$	0.77
50	Brak Popr.	9480	29.21	25.99	$2.76 \cdot 10^{-3}$	0.78
100	Znane Opt.*	10000	408.20	44.02	$4.40 \cdot 10^{-3}$	1.03
100	Brak Popr.*	10000	304.12	42.14	$4.21 \cdot 10^{-3}$	1.00

*Algorytm osiągnął limit iteracji.

Tabela 10: Wyniki dla funkcji Rosenbrocka (Swarm Size = 1000, wątki=4)

Wymiar (n)	Kryterium	$i_{max,4}$	Najlepsze $f(x)$	$T_{thr,4}$ [s]	$T_{thr,4}/i_{max,4}$ [s]	$S(n, 4)$
10	Znane Opt.	7379	$5.64 \cdot 10^{-8}$	16.85	$2.69 \cdot 10^{-3}$	0.15
10	Brak Popr.	3844	$2.24 \cdot 10^{-4}$	11.52	$2.86 \cdot 10^{-3}$	0.14
50	Znane Opt.*	10000	57.83	32.94	$3.29 \cdot 10^{-3}$	0.65
50	Brak Popr.	9470	407.16	33.44	$3.54 \cdot 10^{-3}$	0.61
100	Znane Opt.*	10000	2112.82	42.59	$4.26 \cdot 10^{-3}$	1.07
100	Brak Popr.*	10000	2096.77	34.31	$3.43 \cdot 10^{-3}$	1.23

*Algorytm osiągnął limit iteracji.

CUDA



```
r1 = cp.random.rand(swarm_size, n_dim)
r2 = cp.random.rand(swarm_size, n_dim)

velocities = calculate_velocities(velocities, positions,
                                   pbest_positions, gbest_position, w, c1, c2, r1, r2)

positions += velocities
np.clip(positions, low, high, out=positions)
```



CuPy

Źródło : <https://github.com/cupy/cupy>

Definicja jądra:



```
calculate_velocities = cp.ElementwiseKernel(

    'float64 velocity, float64 position, float64 best_local, float64 best_global, '
    'float64 w, float64 c1, float64 c2, float64 r1, float64 r2',

    'float64 new_velocity',

    'new_velocity = w * velocity + c1 * r1 * (best_local - position) '
    '+ c2 * r2 * (best_global - position)',

    'calculate_velocities')
```

CUDA

Tabela 11: Wyniki dla funkcji Schwefela (Swarm Size = 1000)

Wymiar (n)	Kryterium	$i_{max,cuda}$	Najlepsze $f(x)$	T_{cuda} [s]	$T_{cuda}/i_{max,cuda}$ [s]	$S(n)$
10	Znane Opt.	157	$1.27 \cdot 10^{-4}$	0.62	$3.95 \cdot 10^{-3}$	0.18
10	Brak Popr.	196	$1.27 \cdot 10^{-4}$	0.54	$2.76 \cdot 10^{-3}$	0.21
50	Znane Opt.*	10000	8066.76	60.56	$6.05 \cdot 10^{-3}$	0.51
50	Brak Popr.	384	8066.76	2.17	$5.64 \cdot 10^{-3}$	0.51
100	Znane Opt.*	10000	18703.25	49.16	$4.91 \cdot 10^{-3}$	1.13
100	Brak Popr.	833	18703.25	2.90	$3.49 \cdot 10^{-3}$	1.67

*Algorytm osiągnął limit iteracji nie znajdując optimum z zadaną dokładnością.

Tabela 12: Wyniki dla funkcji Rosenbrocka (Swarm Size = 1000)

Wymiar (n)	Kryterium	$i_{max,cuda}$	Najlepsze $f(x)$	T_{cuda} [s]	$T_{cuda}/i_{max,cuda}$ [s]	$S(n)$
10	Znane Opt.*	10000	$2.53 \cdot 10^{-6}$	61.17	$61.17 \cdot 10^{-3} s$	0.07
10	Brak Popr.	191	5.42	1.99	1.03 s	0.08
50	Znane Opt.*	10000	$9.44 \cdot 10^{-3}$	41.01	$41.01 \cdot 10^{-3} s$	0.52
50	Brak Popr.*	10000	$9.44 \cdot 10^{-3}$	38.15	$38.15 \cdot 10^{-3}$	0.57
100	Znane Opt.*	10000	10108.10	43.42	$43.42 \cdot 10^{-3} s$	1.05
100	Brak Popr.*	10000	10108.10	38.68	$38.68 \cdot 10^{-3} s$	1.09

*Algorytm osiągnął limit iteracji.

MPI



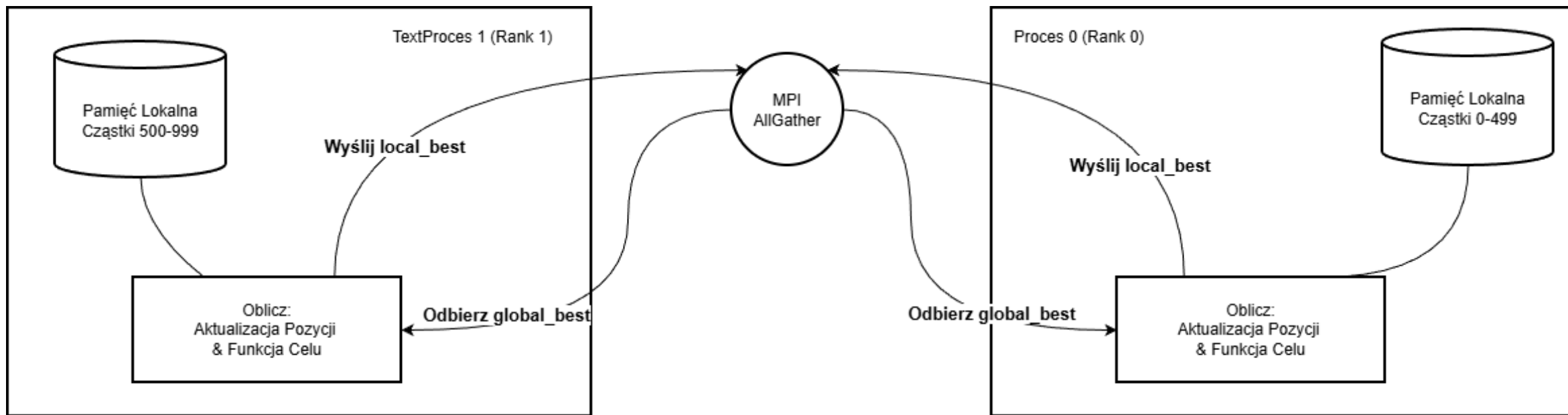
```
mpiexec -n 4 python -m tests.mpi_test
```



```
comm = MPI.COMM_WORLD  
rank = comm.Get_rank()  
size = comm.Get_size()
```



```
all_bests = comm.allgather((local_best_val, local_best_pos))  
best_val_global, best_pos_global = min(all_bests, key=lambda x: x[0])
```



MPI

Tabela 13: Wyniki dla funkcji Schwefela (Swarm Size = 1000)

Wymiar (n)	Kryterium	$i_{max,mpi}$	Najlepsze $f(x)$	$T_{mpi,2}$ [s]	$T_{mpi,4}$ [s]	$S(n, 2)$	$S(n, 4)$
10	Znane Opt.	120	$1.27 \cdot 10^{-4}$	0.14	0.06	0.62	1.44
10	Brak Popr.	161	$1.27 \cdot 10^{-4}$	0.12	0.65	0.79	0.15
50	Znane Opt.*	10000	6726.75	19.17	20.45	1.60	1.50
50	Brak Popr.	357	6726.75	0.59	0.58	1.75	1.78
100	Znane Opt.*	10000	11854.13	38.22	36.52	1.45	1.52
100	Brak Popr.	1024	11854.13	3.83	3.87	1.56	1.54

*Algorytm osiągnął limit iteracji nie znajdując optimum z zadaną dokładnością.

Tabela 14: Wyniki dla funkcji Rosenbrocka (Swarm Size = 1000)

Wymiar (n)	Kryterium	$i_{max,mpi}$	Najlepsze $f(x)$	$T_{mpi,2}$ [s]	$T_{mpi,4}$ [s]	$S(n, 2)$	$S(n, 4)$
10	Znane Opt.*	10000	5.42	3.58	5.59	1.15	0.74
10	Brak Popr.	191	5.42	0.09	0.11	0.89	0.73
50	Znane Opt.*	10000	115.20	13.10	13.88	1.64	1.55
50	Brak Popr.	9003	115.21	10.83	13.11	1.80	1.48
100	Znane Opt.*	10000	119.96	37.31	31.11	1.22	1.46
100	Brak Popr.*	10000	119.96	36.27	29.20	1.16	1.44

*Algorytm osiągnął limit iteracji.

Wnioski końcowe

- **Równoległe nie zawsze oznacza szybsze**
- **Kluczem do wydajności jest dopasowanie modelu równoległości do charakterystyki i rozmiaru przetwarzanych danych.**
- **Dywagacje teoretyczne czy nieprzemyślane zrównoleglanie zwykle nie przynoszą pozytywnych skutków, kiedy w grę wchodzi optymalizacje przygotowane przez autorów bibliotek, interpreterów, kompilatorów czy producentów sprzętu na którym wykonuje się obliczenia, inne warunki mogą przynieść inne wyniki.**

Dziękujemy za uwagę